Rapport Projet E-hotel

Groupe 15

Rayan Adan Ali - 300307261
Sami Diouf -300296671
Abdul Disu - 300367730

1. SGBD Utilisé

Le Système de Gestion de Base de Données (SGBD) retenu pour ce projet est PostgreSQL. Ce choix est justifié par sa robustesse, sa gestion avancée des contraintes d'intégrité (clés étrangères, domaines) et sa capacité à gérer efficacement les déclencheurs (triggers) et les vues demandés dans les spécifications.

2. Technologie utilisé

L'application **e-Hôtels** utilise une architecture client-serveur moderne pour assurer une gestion fluide des données. Voici les technologies clés :

Backend : Développé en **Java (JSP)**, il gère toute la logique métier, comme la recherche de chambres et la conversion des réservations en locations. La connexion à la base de données est assurée par l'API **JDBC**.

Serveur d'application : L'ensemble est hébergé sur **Apache Tomcat**, qui traite les requêtes entre les utilisateurs et le serveur.

Frontend : L'interface est construite en **HTML5** et **CSS3**. Elle propose des formulaires et des listes déroulantes pour permettre aux employés et aux clients de gérer les données facilement, sans aucune commande SQL.

Base de données : Le système s'appuie sur **PostgreSQL** pour stocker les informations. Les requêtes SQL, les index, les vues et les triggers y sont directement intégrés pour garantir la performance et l'archivage automatique.

3. Guide d'Installation

Pour déployer l'application sur un nouvel environnement, suivez ces étapes :

Étape 1 : Configuration de la Base de Données

1. Ouvrez **pgAdmin 4** et connectez-vous à votre serveur PostgreSQL.
2. Créez une nouvelle base de données nommée `postgres` (ou utilisez celle par défaut).
3. Ouvrez l'outil de requête (**Query Tool**) sur cette base.
4. Copiez et exécutez le contenu du fichier `database.sql` (fourni dans le dossier SQL) pour créer les tables, index, vues et triggers.

Étape 2 : Configuration du Serveur Tomcat

1. Installez **Apache Tomcat** (version 9 ou supérieure).
2. Placez les fichiers de l'application (`index.html`, `recherche.jsp`, `employe.jsp`, `admin.jsp`) dans le dossier `webapps/Projet_eHotels/`.
3. Assurez-vous que le pilote JDBC de PostgreSQL (`postgresql-42.x.x.jar`) est présent dans le dossier `lib/` de Tomcat.

Étape 3 : Lancement

1. Démarrez le serveur Tomcat.
2. Accédez à l'interface via votre navigateur à l'adresse : `http://localhost:8080/Projet_eHotels/index.html`

4. DDL (Data Definition Language) - Création de la Base de Données

Le script de définition de données (DDL) complet est fourni en annexe dans le fichier `structure_DB.sql`. Ce script a été généré via pgAdmin et garantit la reproductibilité exacte de la structure de la base de données.

La conception du schéma relationnel repose sur plusieurs piliers techniques afin d'assurer l'intégrité, la performance et l'historisation des données :

A. Tables et Contraintes d'intégrité La base de données contient 8 tables principales (`hotel_chain`, `hotel`, `hotel_room`, `customer`, `employee`, `user`, `reservation`, `reservation_archive`). L'intégrité référentielle est assurée par un système strict de clés primaires et de clés étrangères. Par exemple, la suppression d'un client entraînera la suppression en cascade de ses réservations (`ON DELETE CASCADE`), tandis que la suppression d'un employé au comptoir mettra simplement son identifiant à `NULL` dans l'historique des réservations (`ON DELETE SET NULL`), préservant ainsi la trace de la transaction. Des contraintes de domaine ont également été ajoutées, telles qu'une vérification sur les dates de réservation : `CONSTRAINT check_dates CHECK (end_date > start_date)`.

B. Indexation pour l'optimisation des performances Afin de répondre aux exigences de rapidité de l'interface utilisateur, quatre index B-Tree ont été implémentés sur les colonnes les plus sollicitées lors des recherches :

1. `idx_hotel_city` : Accélère le filtrage des hôtels par zone géographique.
2. `idx_hotel_room_price` : Optimise les recherches de chambres basées sur le budget maximum.
3. `idx_reservation_dates` : Rend la vérification des conflits de dates quasi instantanée lors des check-ins et réservations.
4. `idx_employee_role` : Facilite la génération des listes d'employés autorisés (Gérants et Réceptionnistes).

C. Vues SQL (Views) L'interface et les requêtes analytiques s'appuient sur deux vues précompilées :

- `vue_capacite_totale_hotel` : Calcule dynamiquement la somme des capacités de toutes les chambres pour chaque hôtel.
- `vue_chambres_dispo_par_zone` : Permet de connaître en temps réel le nombre exact de chambres libres dans une ville donnée, en excluant automatiquement celles dont le statut est "OCCUPÉ".

D. Déclencheurs et Fonctions (Triggers) La logique métier a été partiellement déportée vers le SGBD à l'aide de fonctions PL/pgSQL :

- `check_reservation_dates()` : Un trigger qui bloque toute tentative d'insertion si la date de départ est antérieure ou égale à la date d'arrivée.
- `archive_deleted_reservation()` : Un trigger crucial pour la persistance des données. Lors de l'annulation d'une réservation, ce déclencheur insère automatiquement une copie de la ligne supprimée dans la table `reservation_archive`avec un horodatage (`deleted_at`), respectant ainsi la contrainte d'historisation imposée par les spécifications.